

PRACTICAL_10

1. Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Commands:

```
root@SRIVALLI:~# echo "Operating Systems Lab" > file1.txt
root@SRIVALLI:~# ls -li file1.txt
3373 file1.txt
root@SRIVALLI:~# stat file1.txt
  File: file1.txt
  Size: 22          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 3373          Links: 1
Access: (0644/-rw-r--r--)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2026-08-29 08:13:19.288800407 +0000
Modify: 2026-09-27 05:26:42.261888024 +0000
Change: 2026-09-27 05:26:42.261888024 +0000
 Birth: 2026-08-29 08:12:38.051284997 +0000
root@SRIVALLI:~# find . -inum 123456
root@SRIVALLI:~# ln file1.txt hardlink.txt
root@SRIVALLI:~# ls -li file1.txt hardlink.txt
3373 file1.txt 3373 hardlink.txt
root@SRIVALLI:~# ln -s file1.txt softlink.txt
root@SRIVALLI:~# cat hardlink.txt
cat softlink.txt
Operating Systems Lab
Operating Systems Lab
```

Report:

In this experiment, inode information was examined using the `ls -li`, `stat`, and `find` commands. A hard link and a symbolic link were created for a file. The hard link shared the same inode number as the original file, while the symbolic link had a different inode number. This experiment helps in understanding inode allocation, file linking, and file system organization in Linux.

2. Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

CODE:

```

GNU nano 7.2 mmap_file.c *
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <string.h>

int main()
{
    int fd;
    char *data;

    fd = open("sample.txt", O_RDWR | O_CREAT, 0666);

    write(fd, "Hello Linux", 11);

    data = mmap(NULL, 11,
                PROT_READ | PROT_WRITE,
                MAP_SHARED,
                fd, 0);

    printf("File Content: %s\n", data);

    strcpy(data, "Hello mmap!");

    munmap(data, 11);
    close(fd);

    return 0;
}

```

RESULT:

```

root@SRIVALLI:~# nano mmap_file.c
root@SRIVALLI:~# gcc mmap_file.c -o mmap_file
root@SRIVALLI:~# ./mmap_file
File Content: Hello Linux
root@SRIVALLI:~# cat sample.txt
Hello mmap!root@SRIVALLI:~#

```

REPORT:

In this experiment, the `mmap()` system call was used to perform file reading and writing through memory mapping. The file contents were accessed directly through memory instead of using `read()` and `write()` system calls. Memory-mapped I/O provides faster access for large files because it reduces the number of data copies between user space and kernel space. However, it is slightly more complex to implement compared to traditional file I/O operations. This experiment helps in understanding memory-mapped file handling and its advantages in Linux systems.